

Projektdokumentation: Implementierung einer End-to-End BI-Lösung

Vom operationalen Datenbanksystem (OLTP) zum analytischen Data Warehouse (DWH)

1. Projekt-Roadmap

Das Hauptziel dieses Projekts war die Transformation von unstrukturierten Verkaufsdaten in eine entscheidungsrelevante Datenstruktur für das Management. Der Prozess wurde in drei Hauptebenen unterteilt:

- **Rohdatenebene:** Nutzung von CSV-Dateien (Kaggle-Dataset) mit über 500.000 Transaktionen.
- **Operationale Ebene (Retail_OLTP):** Ein hochgradig normalisiertes relationales Datenbankmodell (3NF) zur präzisen täglichen Datenspeicherung.
- **Analytische Ebene (Retail_DWH):** Ein Data Warehouse im Sternschema (Star Schema) für hochperformante Abfragen und Business-Reporting.

2. Beschreibung der Ebenen und Entitäten

A) Operationale Datenbank (Business DB - OLTP)

Das Design konzentriert sich auf die **Datenintegrität** und die Minimierung von Redundanz:

- **Store & Employee:** Verwaltung von 10 Filialen in 5 Regionen und 20 Mitarbeitern zur klaren Verantwortungszuweisung.
- **Products & Customers:** Zentrale Speicherung von Stammdaten, um Dubletten zu vermeiden.
- **SalesHeaders & Details:** Implementierung eines Master-Detail-Modells zur Trennung von allgemeinen Rechnungsinformationen (Datum, Kunde) und spezifischen Positionsdaten (Menge, Einzelpreis).

B) Data Warehouse (DWH - OLAP)

Hier wurde die Struktur für analytische Zwecke denormalisiert und in ein **Sternschema** überführt:

- **Faktentabelle (Sales_Fact):** Das Zentrum des Modells, das Surrogate Keys und messbare Kennzahlen (Quantity, UnitPrice, TotalPrice) enthält.
- **Dimensionen:** Fünf Schlüsseldimensionen (Time, Product, Customer, Store, Employee) ermöglichen multidimensionale Analysen.

3. Technische Herausforderungen & Engineering Solutions (Lessons Learned)

Herausforderung 1: Kategorisierung unstrukturierter Daten (Mapping)

- **Problem:** Den Rohdaten fehlte eine Spalte für "Produktkategorien", die für das Management-Reporting essenziell ist.
- **Lösung:** Erstellung einer Mapping-Logik in Python. Basierend auf Schlüsselwörtern in der Spalte Description (z. B. "Bag" oder "Kitchen") wurden die Produkte logischen Kategorien zugewiesen.

Herausforderung 2: Treiberfehler und Datumsformate (Data Type Mismatch)

- **Problem:** Beim Datentransfer via ODBC trat der Fehler **HYC00** auf, da komplexe Python-Zeitformate nicht direkt in SQL konvertiert werden konnten.
- **Lösung:** Im ETL-Skript wurden alle Zeitstempel mit der Methode `.strftime()` in Standard-Strings umgewandelt. Zudem wurde das Feld Hour separat extrahiert, um stündliche Verkaufsanalysen zu ermöglichen.

Herausforderung 3: Historisierung von Preisen (SCD Typ 2)

- **Problem:** Preisänderungen im Quellsystem würden historische Finanzberichte verfälschen (Retrograde Fehler).
- **Lösung:** Implementierung von **Slowly Changing Dimension (SCD) Typ 2** in der Tabelle Product_Dim. Durch die Felder IsActive, Start_Date und End_Date bleibt die Preishistorie vollständig erhalten.

Herausforderung 4: Performance-Optimierung (Storage & Speed)

- **Problem:** Die Berechnung von TotalPrice bei über 500.000 Datensätzen während der Abfrage verlangsamte das System.
- **Lösung:** Definition von TotalPrice als **Persisted Computed Column**. Das Ergebnis wird einmalig beim Laden berechnet und physisch gespeichert, was die Lese-Performance massiv steigert.

4. Änderungsmanagement-Strategie (SCD-Strategie)

- **SCD Typ 1:** Anwendung auf **Store** und **Employee**; hier wird nur der aktuelle Stand benötigt (z. B. aktueller Standort eines Mitarbeiters).
- **SCD Typ 2:** Anwendung auf **Product**; unverzichtbar für Finanzanalysen, um Verkäufe stets mit dem zum Transaktionszeitpunkt gültigen Preis zu verknüpfen.

5. Fazit und Highlights für die Präsentation

1. **Datenqualität:** Ungeordnete CSV-Daten wurden in ein konsistentes, bereinigtes Sternschema überführt.
2. **Business Intelligence:** Das System beantwortet komplexe Fragen wie: *"Was ist die profitabelste Verkaufsstunde in den Filialen der Region Bayern?"* in Echtzeit.
3. **Automatisierung:** Durch Python (Pandas & PyODBC) wurde eine robuste ETL-Pipeline geschaffen, die täglich automatisierte Batch-Läufe ermöglicht.

Empfehlung für die Zukunft: Bei steigendem Datenvolumen sollte die Datenbereinigung stets in Python vor dem SQL-Bulk-Insert erfolgen, um Datenbank-Locks zu vermeiden.

Datum: Mai 2026